Practical 9: Feature Extraction and Transformation## Practical Information| Attribute | Details ||----------|---------|| **Practical Number** | 9 || **Title** | Feature Extraction and Transformation || **Duration** | 2 Hours || **Course Outcomes (COs)** | CO1, CO3 || **Description |** Extract and transform features from raw data for machine learning models. |## Key Topics Covered- Polynomial features- Text extraction- TF-IDF- Word embeddings- Feature pipelines---## Instructions for Students1. **Complete all tasks** outlined in this practical2. **Write your code** in the cells provided below3. **Add comments** to explain your code4. **Include proper output** (prints, visualizations, etc.)5. **Submit your completed notebook** before the deadline---## Your Work Starts HereUse the cell(s) below to write your implementation:

```
# Practical 9: Feature Extraction and Transformation

import pandas as pd
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score

# Load dataset
data = fetch_california_housing(as_frame=True)
X = data.data
y = data.target

# Create a new ratio feature
X['RoomsPerPerson'] = X['AveRooms'] / X['Population']

# Select top 3 features for polynomial expansion
top_features = ['MedInc', 'HouseAge', 'RoomsPerPerson']

# Polynomial Features (degree 2)
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly_raw = poly.fit_transform(X[top_features])

# Convert to DataFrame
poly_cols = poly.get_feature_names_out(top_features)
X_poly = pd.DataFrame(X_poly_raw, columns=poly_cols)

# Join with original features
X_final = pd.concat([X.reset_index(drop=True), X_poly.reset_index(drop=True)], axis=1)
X_final = X_final.loc[:, ~X_final.columns.duplicated()]

print("Total features after polynomial expansion:", X_final.shape[1])
print("Polynomial features created:", list(poly_cols))

# Model evaluation
score_base = cross_val_score(LinearRegression(), X, y, cv=5, scoring='r2').mean()
score_poly = cross_val_score(LinearRegression(), X_final, y, cv=5, scoring='r2').mean()

print(f"Baseline R² Score: {score_base:.4f}")
print(f"Polynomial Features R² Score: {score_poly:.4f}")
```

```
Total features after polynomial expansion: 15
Polynomial features created: ['MedInc', 'HouseAge', 'RoomsPerPerson', 'MedInc^2', 'MedInc HouseAge', 'MedInc RoomsPerPerson', 'H
Baseline R² Score: 0.5533
Polynomial Features R² Score: 0.5231
```

```
# Practical 9 - Code 2: Feature Transformations (Log, Sqrt, Binning)
```

```
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_california_housing

# Load dataset
data = fetch_california_housing(as_frame=True)
X = data.data.copy()

# Apply transformations
X['Population_Log'] = np.log1p(X['Population'])
X['AveRooms_Sqrt'] = np.sqrt(X['AveRooms'])

# Binning continuous feature into categories
X['HouseAge_Bin'] = pd.cut(X['HouseAge'], bins=[0, 20, 40, 60], labels=['Low', 'Medium', 'High'])

print("Transformed Features Preview:")
print(X[['Population', 'Population_Log', 'AveRooms', 'AveRooms_Sqrt', 'HouseAge', 'HouseAge_Bin']].head())

# Check skewness reduction
print("\nSkewness before log:", data.data['Population'].skew())
print("Skewness after log:", X['Population_Log'].skew())
```

```
Transformed Features Preview:
   Population  Population_Log  AveRooms  AveRooms_Sqrt  HouseAge HouseAge_Bin
0       322.0        5.777652  6.984127       2.642750      41.0         High
1      2401.0        7.784057  6.238137       2.497626      21.0       Medium
2       496.0        6.208590  8.288136       2.878912      52.0         High
3       558.0        6.326149  5.817352       2.411919      52.0         High
4       565.0        6.338594  6.281853       2.506363      52.0         High

Skewness before log: 4.93585822672712
Skewness after log: -1.0440866961519721
```

```
# Practical 9 - Code 3: Text Feature Extraction using TF-IDF Pipeline

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Sample text dataset
texts = [
    "Machine learning is amazing",
    "I love studying data science",
    "Deep learning is part of machine learning",
    "Data science uses machine learning",
    "I enjoy coding in Python"
]
labels = [1, 1, 1, 1, 0]  # 1 = ML related, 0 = Not ML related

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(texts, labels, test_size=0.2, random_state=42)

# Create pipeline: TF-IDF + Classifier
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer()),
    ('model', LogisticRegression())
])

# Train model
pipeline.fit(X_train, y_train)

# Predictions
y_pred = pipeline.predict(X_test)

print("Predictions:", y_pred)
print("Actual     :", y_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Predictions: [1]
Actual     : [1]
Accuracy: 1.0
```